

Fiscal Agent

Architettura, scelte progettuali e integrazioni

Progetto: Assistente fiscale AI — Demo
Aprile 2026

Stack: Python · LlamaIndex · OpenAI · HuggingFace

Data:

Overview

Fiscal Agent è un assistente conversazionale AI che risponde in tempo reale alle domande fiscali dei clienti Fiscozen, personalizzando ogni risposta con i dati specifici del cliente e il contenuto della knowledge base aziendale.

I consulenti e i clienti Fiscozen si trovano spesso a porre domande ricorrenti su regime forfettario, contributi previdenziali, fatturazione e scadenze. Rispondere manualmente a ogni richiesta richiede tempo e comporta il rischio di incoerenze. Il Fiscal Agent risolve questo problema automatizzando le risposte di primo livello, mantenendo il tono di voce Fiscozen e delegando al Customer Success Consultant solo i casi complessi.

Obiettivi specifici



Risposte precise e affidabili

L'agente usa esclusivamente la knowledge base aziendale come fonte. Non inventa dati fiscali.



Personalizzazione per cliente

Ogni risposta tiene conto del regime fiscale, del fatturato e del consulente assegnato al cliente.



Tono di voce coerente

Il sistema prompt applica le linee guida Fiscozen a ogni risposta, senza deroghe.



Escalation intelligente

Quando la domanda è ambigua o rischiosa, l'agente rimanda al Customer Success Consultant assegnato.

Perimetro della demo

La demo opera su un campione di 4 clienti fittizi (Mario Rossi, Giulia Verdi, Luca Martini, Elena Riva) e due documenti fiscali rappresentativi. L'interfaccia è a riga di comando (CLI), con input via terminale o flag.

Scelte progettuali

Il sistema combina tre componenti distinti — recupero del contesto fiscale (RAG), lookup del profilo cliente, generazione del linguaggio — orchestrati in un singolo prompt strutturato. Ogni scelta è motivata da un requisito preciso.

RETRIEVAL

RAG con LlamaIndex

COSA FA

I documenti fiscali (`.docx`) vengono indicizzati in un *VectorStoreIndex*. Ad ogni domanda, il sistema recupera i 4 chunk più semanticamente rilevanti (512 token, overlap 64) e li inserisce nel prompt.

△ PERCHÉ

Il modello LLM non può memorizzare l'intero corpus fiscale nel contesto. Il RAG garantisce che la risposta sia basata su fonti verificabili, riducendo le allucinazioni.

EMBEDDING

HuggingFace locale — paraphrase-multilingual-MiniLM-L12-v2

COSA FA

Gli embedding vettoriali dei documenti e delle query vengono calcolati localmente, senza chiamate API esterne. Il modello scelto è ottimizzato per 50+ lingue, incluso l'italiano.

△ PERCHÉ

Azzera i costi di embedding (nessuna quota OpenAI consumata per il RAG). Il modello multilingue gestisce correttamente termini fiscali italiani senza fine-tuning.

GENERAZIONE

OpenAI GPT — temperatura 0.2

COSA FA

Il modello GPT riceve un prompt strutturato (system + user) e genera la risposta finale in italiano. La temperatura è impostata a 0.2 per massimizzare la coerenza delle risposte.

△ PERCHÉ

Una temperatura bassa riduce la variabilità e le risposte creative non pertinenti, che in ambito fiscale possono essere fuorvianti. La qualità del linguaggio di GPT supera i modelli open-source per l'italiano.

PROMPT DESIGN

Separazione system prompt / user prompt

COSA FA

Il **system prompt** contiene il tono di voce (fisso, caricato da file) e le regole operative. Il **user prompt** è costruito dinamicamente con: domanda, dati del cliente, contesto fiscale dal RAG.

⚠ PERCHÉ

Separare le istruzioni permanenti dai dati dinamici rende il tono di voce inviolabile e semplifica l'aggiornamento indipendente di ognuno dei tre blocchi senza toccare il codice.

DATI CLIENTE

Excel + Pandas — lookup per nome e cognome

COSA FA

I dati del cliente (regime, cassa, consulente, fatturato, data apertura P.IVA) sono letti da un file `.xlsx` via Pandas. Il DataFrame viene caricato in memoria una sola volta e riusato per ogni query.

⚠ PERCHÉ

Excel è già il formato nativo usato dai team operativi Fiscozen. Eliminare un layer di database riduce la complessità infrastrutturale nella fase demo, mantenendo un'interfaccia familiare per il team.

ESCALATION

Escalation al consulente — regola nel system prompt

COSA FA

Il system prompt istruisce il modello a rimandare al Customer Success Consultant assegnato (letto dai dati del cliente) quando la domanda è complessa, ambigua o a rischio per il cliente.

⚠ PERCHÉ

Gestire l'escalation via prompt — anziché con logica applicativa — permette di affinare la soglia di delegazione modificando solo testo, senza deploy di codice. Il nome del consulente è sempre personalizzato.

Componenti e dipendenze

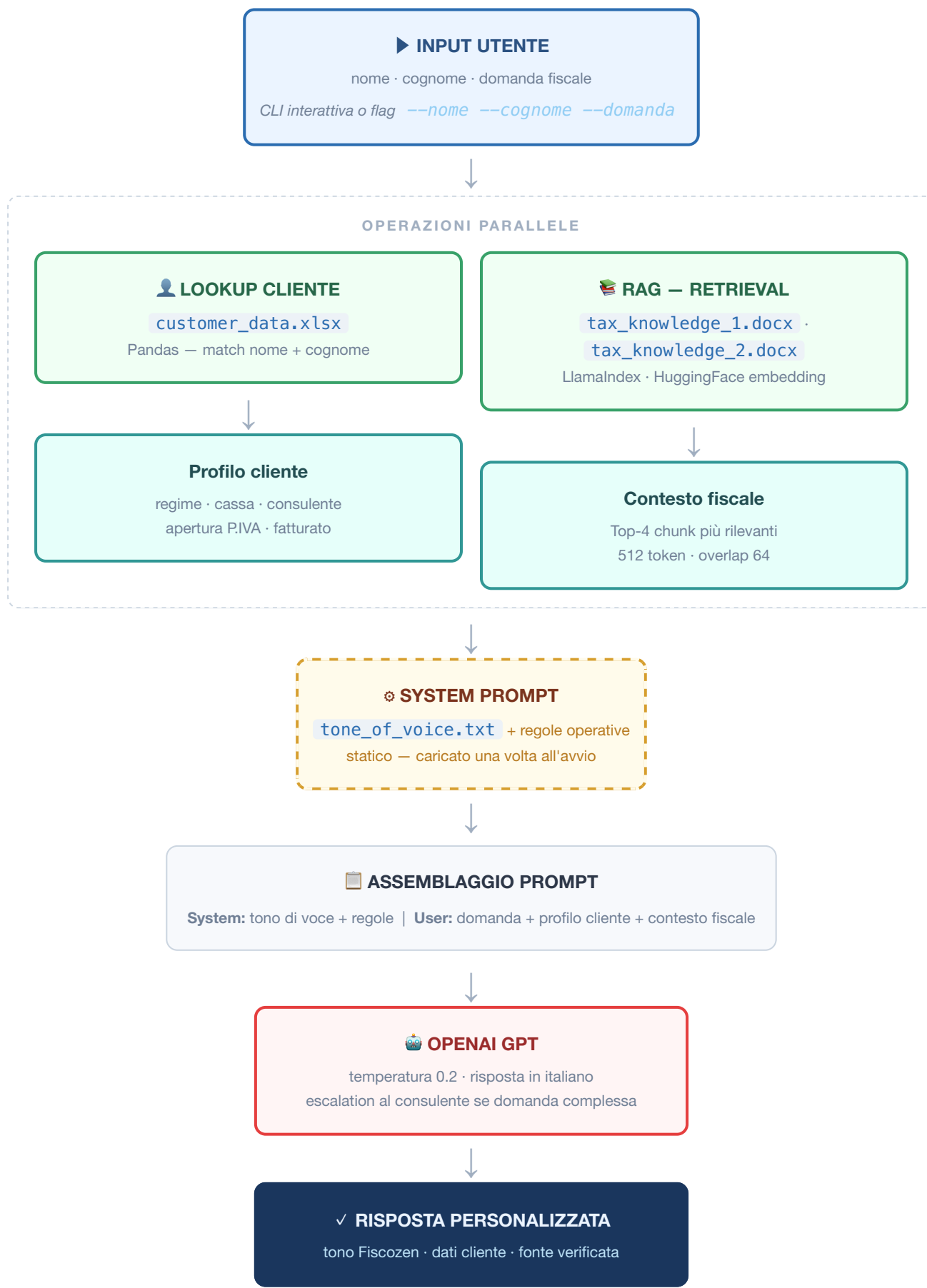
Componente	Libreria / Servizio	Tipo	Ruolo nel sistema
LLM — Generazione	<code>openai</code> <code>>= 1.0</code>	API esterna	Genera la risposta finale in italiano. Riceve il prompt strutturato e restituisce il testo per il cliente.
RAG — Orchestrazione	<code>llama-index-core</code> <code>>= 0.10</code>	Locale	Gestisce indicizzazione vettoriale e retrieval. Costruisce il <code>VectorStoreIndex</code> dai documenti e risponde alle query con i chunk più rilevanti.
Embedding — Vettorializzazione	<code>llama-index-embeddings-huggingface-sentence-transformers</code>	Locale	Converte testo in vettori numerici. Modello: <code>paraphrase-multilingual-MiniLM-L12-v2</code> . Gira in locale, senza chiamate API.
Lettura documenti	<code>llama-index-readers-file-docx2txt</code> , <code>python-docx</code>	File locale	Legge e analizza i file <code>.docx</code> della knowledge base fiscale. Converte il contenuto in nodi testuali per l'indicizzazione.
Dati cliente	<code>pandas</code> <code>>= 2.0</code> <code>openpyxl</code> <code>>= 3.1</code>	File locale	Legge <code>customer_data.xlsx</code> . Esegue il lookup per nome e cognome. Formatta il profilo cliente da inserire nel prompt.
Configurazione ambiente	<code>python-dotenv</code> <code>>= 1.0</code>	Config	Carica <code>OPENAI_API_KEY</code> dal file <code>.env</code> locale. Mantiene le credenziali fuori dal codice sorgente.

Parametri chiave del sistema

Modello LLM	GPT (OpenAI) — via <code>openai</code> SDK
Modello embedding	<code>paraphrase-multilingual-MiniLM-L12-v2</code> — locale, gratuito
Chunk size	512 token con overlap di 64 token
Top-k retrieval	4 chunk — i più semanticamente vicini alla domanda
Temperatura LLM	0.2 — bassa variabilità, risposte coerenti

Diagramma del flusso

Il diagramma mostra il percorso di ogni richiesta, dall'input dell'utente fino alla risposta generata dal modello.



 Input /
Output

 Elaborazione
dati

 Configurazione
statica

 Chiamata LLM (API
esterna)

 Output
finale